

MAKING FISHER WORK: TRAIN-TIME COMPRESSION FOR 3D GAUSSIAN SPLATTING

P G Arun Madhav *Chandra Sekhar Seelamantula*

Department of Electrical Engineering, Indian Institute of Science, Bangalore

ABSTRACT

3D Gaussian Splatting (3D-GS) enables high-quality novel view synthesis with real-time rendering, but typically requires millions of Gaussian primitives, resulting in substantial memory and storage footprints. Most existing compression methods rely on heuristic pruning or generic quantization, which are ineffective at preventing redundancy accumulation due to unstable early-stage optimization.

In this work, we propose a compact training framework for 3D-GS that enables reliable compression during optimization. Our method employs progressive pyramidal training, allowing Gaussian primitives to stabilize under coarser scene representations before incorporating fine-scale appearance details. This formulation enables reliable estimation of Fisher Information Matrix-based sensitivity scores during training, which are subsequently used to prune redundant primitives. To further reduce storage, we introduce a correlation-aware quantization strategy.

Experiments on standard novel view synthesis benchmarks demonstrate that our method achieves an average reduction of 95% in storage footprint while preserving high-quality rendering.

Index Terms— 3D Reconstruction, Quantization, Score-based Pruning.

1. INTRODUCTION

3D Gaussian Splatting (3D-GS) [1] represents scenes using Gaussian primitives, enabling efficient rasterization and real-time rendering while achieving competitive visual fidelity compared to previously developed Neural Radiance Field-based approaches [2, 3]. Despite its efficiency at inference, 3D-GS exhibits a fundamental scalability challenge: high-quality reconstructions typically require millions of Gaussian primitives, resulting in substantial memory consumption and a large storage footprint. A natural direction for addressing this challenge is to reduce redundancy in the Gaussian representation. Recent work has explored pruning and parameter compression strategies, either by removing redundant primitives or by quantizing Gaussian parameters [4–10]. However, pruning decisions are particularly challenging during training, as early optimization stages are often unstable, leading to premature removal of necessary primitives. Similarly, parameter

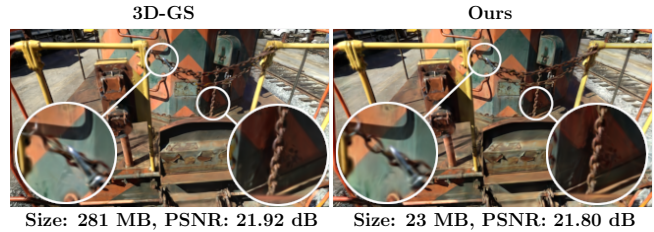


Fig. 1. We reduce the storage footprint of the 3D Gaussian Splatting reconstruction of the Tanks&Temples – *train* scene from 281 MB to 23 MB, achieving 92.24% compression while preserving fine-scale details.

quantization alone cannot sufficiently reduce storage when the underlying representation remains highly redundant.

The aforementioned limitations point to a key observation: effective compression of 3D-GS requires not only reducing parameter precision, but also controlling how redundancy emerges throughout training. Motivated by this observation, we propose a training framework that explicitly regulates the introduction of image detail, enables reliable identification of redundant primitives during optimization, and performs correlation-aware quantization of Gaussian parameters to achieve efficient compression. We summarize our contributions as follows:

- We develop a unified training framework that enables reliable Fisher Information Matrix-based pruning during training via progressive pyramidal training.
- We introduce a correlation-aware quantization strategy that leverages the underlying correlation structure of Gaussian parameters to enable efficient storage.

As a consequence, we obtain a compact 3D-GS representation that substantially reduces the storage footprint while preserving rendering quality and real-time performance.

2. RELATED WORK

Recent advances in 3D Gaussian Splatting (3D-GS) have prompted growing interest in improving the compactness and efficiency of scene representations. We briefly review representative methods along these directions.

Progressive training: Progressive training is an efficient strategy to improve the optimization performance of 3D-GS. EAGLES [5] introduces multiscale image representations for accelerated convergence, and Opti3DGS [11] explores frequency-based progressive training. Although these approaches improve optimization stability, they are not designed to regulate redundancy formation or enable compression during training.

Pruning: A significant fraction of Gaussian primitives in 3D-GS are redundant [5], which motivated a line of work focused on primitive pruning. LightGaussian [6] computes a global significance score, and EAGLES [5] estimates the score via accumulated transmittance. PUP [7] introduces a more principled approach by deriving a per-Gaussian score from the Hessian of the $\mathcal{L}_2$ reconstruction loss. While PUP achieves state-of-the-art post hoc pruning results, it is applicable only near convergence.

Quantization: Parameter quantization has been widely explored to reduce the storage footprint of 3D-GS representations. Compact 3D-GS [4] employs learned discrete latent codes, while EAGLES [5] and LightGaussian [6] combine pruning with low-precision quantization. However, most approaches apply generic quantization schemes uniformly across heterogeneous Gaussian parameters, limiting compression efficiency.

In contrast to these prior works, our method enables reliable pruning during training by controlling redundancy formation through progressive pyramidal training, and further improves storage efficiency via correlation-aware quantization tailored to the correlation structure of Gaussian parameters.

3. BACKGROUND

3.1. 3D Gaussian Splatting

3D Gaussian Splatting (3D-GS) [1] is a point-based novel-view synthesis technique that represents a scene as a collection of 3D Gaussian primitives. Each Gaussian primitive $\mathcal{G}_i$ is parameterized by its center $\mu_i \in \mathbb{R}^3$, scale parameters $s_i \in \mathbb{R}^3$, rotation parameters $r_i \in \mathbb{R}^4$, base color $b_i \in \mathbb{R}^3$, view-dependent spherical harmonic coefficients $h_i \in \mathbb{R}^{15 \times 3}$, and opacity $o_i \in \mathbb{R}$. The parameters of the Gaussian primitives are collected as:

$$\mathcal{G} = \{\mathcal{G}_i = \{\mu_i, s_i, r_i, b_i, h_i, o_i\}\}_{i=1}^N, \quad (1)$$

where N is the total number of Gaussian primitives.

Given a set of training views with estimated camera poses $\mathcal{P} = \{\phi_i \in \mathbb{R}^{3 \times 4}\}_{i=1}^K$ and corresponding ground-truth images $\mathcal{I} = \{\mathbf{I}_i \in \mathbb{R}^{H \times W}\}_{i=1}^K$, a differentiable rasterizer projects the primitives observed from pose ϕ onto the image plane to produce a rendered image $\mathbf{I}_{\mathcal{G}}(\phi)$.

The parameters $\mathcal{G}$ are optimized by minimizing a weighted combination of the photometric loss $\mathcal{L}_1$ and the perceptual

similarity metric $\mathcal{L}_{D-SSIM}$ between the rendered image and the corresponding ground-truth image:

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{D-SSIM}, \quad (2)$$

where λ is typically set to 0.2 [1].

3.2. Fisher Information Matrix-based Pruning

To obtain a per-Gaussian sensitivity, PUP [7] considers the $\mathcal{L}_2$ reconstruction error over the set of training views:

$$\mathcal{L}_2 = \frac{1}{2} \sum_{i=1}^K \|\mathbf{I}_{\mathcal{G}}(\phi_i) - \mathbf{I}_i\|_2^2.$$

For a converged 3D-GS model, the $\mathcal{L}_1$ residual in Eq. 2 becomes negligible. In this regime, the Hessian of $\mathcal{L}_2$ is an approximation of the Fisher Information Matrix [12, 13]:

$$\nabla_{\mathcal{G}}^2 \mathcal{L}_2 \approx \sum_{i=1}^K \nabla_{\mathcal{G}} \mathbf{I}_{\mathcal{G}}(\phi_i) \nabla_{\mathcal{G}} \mathbf{I}_{\mathcal{G}}(\phi_i)^{\top}, \quad (3)$$

where $\nabla_{\mathcal{G}}$ is the gradient with respect to the parameters of the Gaussian primitives as defined in Eq. 1.

To obtain per-Gaussian sensitivity, PUP restricts attention to the block-diagonal structure corresponding to individual Gaussian primitives. Let $\mathbf{H}_j$ denote the per-Gaussian block for the Gaussian primitive $\mathcal{G}_j$:

$$\mathbf{H}_j = \sum_{i=1}^K \nabla_{\mathcal{G}_j} \mathbf{I}_{\mathcal{G}}(\phi_i) \nabla_{\mathcal{G}_j} \mathbf{I}_{\mathcal{G}}(\phi_i)^{\top},$$

where $\nabla_{\mathcal{G}_j}$ is the gradient with respect to the parameters of the Gaussian primitive $\mathcal{G}_j$. The block $\mathbf{H}_j$ quantifies the sensitivity of the $\mathcal{L}_2$ reconstruction error to perturbations of the parameters of the Gaussian primitive $\mathcal{G}_j$ in isolation on all training views.

The block $\mathbf{H}_j$ is converted into a scalar sensitivity score U_j by taking the log-determinant:

$$U_j = \log |\mathbf{H}_j|, \quad (4)$$

which provides a scalar measure of the Gaussian primitive's contribution to the rendered images over the training set. Gaussian primitives with lower sensitivity scores are considered redundant and are removed during pruning.

4. METHOD

We present a compact training framework for 3D Gaussian Splatting (3D-GS) that enables reliable Fisher Information Matrix-based pruning during training through progressive pyramidal supervision, and further improves storage efficiency via correlation-aware quantization that leverages inter-channel dependencies of spherical harmonic coefficients. An overview of the framework is shown in Fig. 2(a), resulting in a substantial reduction in storage footprint.

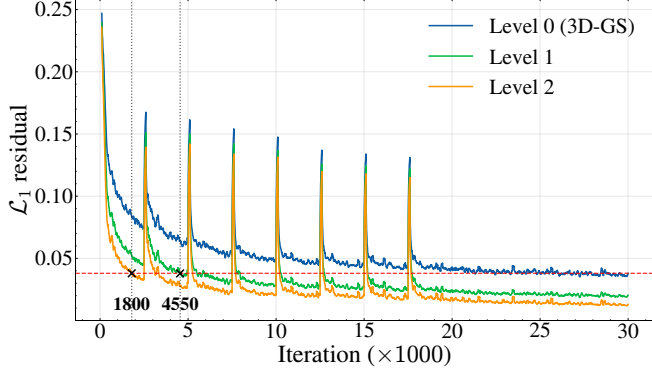


Fig. 3. $\mathcal{L}_1$ residual across iterations for different pyramid levels, where vertical markers denote the first iteration at which each pyramid level achieves a lower $\mathcal{L}_1$ residual than a fully converged 3D-GS model on the Tanks&Temples dataset.

attention to the block-diagonal structure of the Hessian. Let $\mathbf{H}_j$ denote the block corresponding to the parameters of the Gaussian primitive $\mathcal{G}_j$:

$$\mathbf{H}_j = \sum_{i=1}^K \mathbf{J}_{f_\ell} \nabla_{\mathcal{G}_j} \mathbf{I}_G(\phi_i) \nabla_{\mathcal{G}_j} \mathbf{I}_G(\phi_i)^\top \mathbf{J}_{f_\ell}^\top.$$

We define the sensitivity score of Gaussian primitive $\mathcal{G}_j$ as the log-determinant of the corresponding Hessian block $\mathbf{H}_j$ as defined in Eq. 4. This score quantifies the contribution of the corresponding Gaussian primitive to the rendered images under the current pyramid-level supervision over the training set.

Pyramidal Pruning: This interleaving of progressive pyramidal training with FIM-based pruning prevents redundancies formed at coarser stages from propagating into subsequent finer-scale optimization phases. Rather than directly enforcing a compact final representation, this stage-wise pruning acts as a regularization mechanism that limits the accumulation of redundant primitives during training.

As training progresses, removing redundant Gaussian primitives reduces the effective optimization set, thereby improving training efficiency. This prevents redundant structure learned under coarse supervision from biasing subsequent fine-scale optimization, while enabling the model to allocate representational capacity more effectively as higher-frequency details are introduced.

Iterative Pruning: Finally, we analyze the temporal stability of the FIM-based sensitivity scores during training. In the later stages of optimization, as the model approaches convergence, we evaluate the consistency of per-Gaussian sensitivity rankings using Spearman’s rank correlation [15], measured with respect to a fully converged 3D-GS model, as shown in Fig. 4. The consistently high rank agreement indicates that the relative importance ordering of Gaussian primitives remains stable in later training stages.

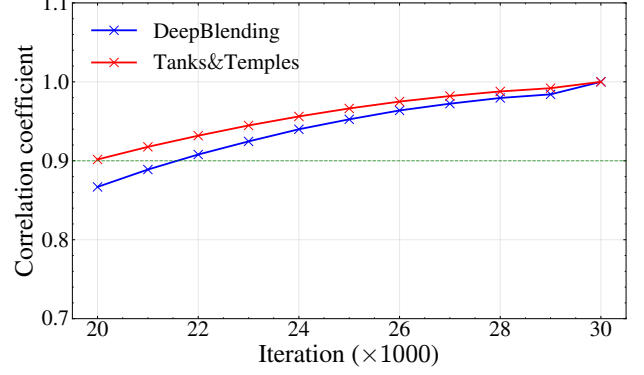


Fig. 4. Spearman’s rank-correlation coefficient between Fisher Information Matrix-based sensitivity rankings at different training iterations and the rankings of a fully converged 3D Gaussian Splatting model.

Motivated by the observed stability, we perform pruning every 3,000 iterations between 21,000 and 27,000 iterations, which empirically provides an effective balance between pruning efficiency and training stability.

4.3. Quantization

High-quality 3D Gaussian Splatting reconstructions often require millions of Gaussian primitives, resulting in substantial storage overhead. In particular, spherical harmonic (SH) coefficients defined in Eq. 1, which model view-dependent appearance, dominate the parameter budget, accounting for over 80% of the per-Gaussian storage for cubic SH degree.

Rather than applying a generic quantization scheme across all Gaussian parameters, we employ correlation-aware quantization strategies: rotation ($\mathbf{r}$) and opacity (o) parameters are decorrelated [5] and are therefore quantized using scalar latents ($\mathbf{r}_\ell, o_\ell$) coupled with a compact linear decoder $\mathcal{D}$. In contrast, spherical harmonic coefficients ($\mathbf{h}$) exhibit strong inter-channel correlation induced by their frequency-domain structure [16]. To exploit this structure, we employ a Discrete Fourier Transform-based linear decoder $\mathcal{D}_{\text{DFT}}$ [17], which maps correlated SH channels into a frequency basis and facilitates efficient latent ($\mathbf{h}_\ell$) compression.

To support gradient-based optimization, quantization is incorporated during training through continuous latent variables combined with Straight-Through Estimator (STE) [18]:

$$\mathbf{h} = \mathcal{D}_{\text{DFT}}(\text{STE}(\mathbf{h}_\ell)), \quad \mathbf{r}, o = \mathcal{D}(\text{STE}(\mathbf{r}_\ell, o_\ell)).$$

STE enables discretization in the forward pass while preserving gradient flow during backpropagation, thereby enabling the quantized latents and corresponding decoders to be jointly optimized with the 3D-GS.

After training, the learned continuous latents are discretized, entropy-coded, and stored together with their corresponding decoders.

Table 1. Metrics on Mip-360, Tanks&Temples, and Deep Blending datasets. Best results in red, second best in orange.

Method	Mip-NeRF 360				Tanks&Temples				Deep Blending			
	PSNR↑	SSIM↑	LPIPS↓	Size (MB)↓	PSNR↑	SSIM↑	LPIPS↓	Size (MB)↓	PSNR↑	SSIM↑	LPIPS↓	Size (MB)↓
3DGS [1]	27.44	0.813	0.218	822.6	23.67	0.844	0.179	452.4	29.48	0.900	0.246	692.5
Compact [4]	26.95	0.797	0.244	26.31	23.33	0.831	0.202	18.97	29.71	0.901	0.257	25.75
EAGLES [5]	27.07	0.801	0.240	59.49	23.14	0.833	0.203	30.18	29.72	0.906	0.249	54.45
LightGaussian [6]	26.90	0.800	0.240	53.96	23.32	0.829	0.204	29.94	29.12	0.895	0.262	45.25
PUP [7]	26.67	0.786	0.271	74.65	22.72	0.801	0.244	43.37	28.85	0.881	0.301	69.98
Ours	27.01	0.801	0.239	32.78	23.38	0.837	0.202	23.93	29.87	0.912	0.248	30.07

5. EXPERIMENTS

5.1. Datasets

We evaluate our method on three widely used real-world benchmarks for 3D Gaussian Splatting: the Mip-NeRF 360 dataset [19], the Tanks&Temples dataset [20], and the Deep Blending dataset [21]. For all experiments, we use the camera pose estimates produced by COLMAP [22] that are provided with the respective datasets to ensure consistency with prior work.

5.2. Implementation Details

Our implementation is built upon the official 3D Gaussian Splatting framework [1]. We employ a three-level pyramidal training schedule, starting from the coarsest level and advancing to finer pyramid levels, with transitions at 5,000 and 10,000 iterations. To compute per-Gaussian sensitivity scores during training, we extend the Fisher rasterizer introduced in PUP [7] to operate under our progressive pyramidal supervision. Gaussian primitives are pruned according to the pyramidal and iterative strategies described in Sec. 4.2, with scene-specific pruning ratios provided in the supplementary material Sec. 8.1.

5.3. Results

In Table 1, we compare our method against prior work that employs score-based pruning and parameter quantization [4–6], as well as the parent method PUP [7]. Our method achieves an average storage reduction of 95% while preserving high-quality rendering. Comparisons of training time and rendering speed (FPS) are provided in the supplementary material Sec. 8.1, along with additional qualitative results.

5.4. Ablation

Table 2 demonstrates that correlation-aware quantization (CAQ) yields a substantial reduction in storage with a moderate drop in reconstruction quality. Progressive pyramidal

Table 2. Ablation study Mip-NeRF 360 *bonsai* scene.

Method	PSNR↑	SSIM↑	LPIPS↓	Primitives↓	Size(MB)↓
3D-GS	32.25	0.946	0.181	1.25 M	294.55
+ CAQ	31.36	0.937	0.216	0.98 M	44.22
+ PPT	31.35	0.936	0.216	0.80 M	36.82
+ PP	30.94	0.933	0.219	0.66 M	30.88
+ IP	30.91	0.932	0.224	0.33 M	15.24
Ours	30.92	0.932	0.224	0.33 M	15.28

training (PPT) further reduces the number of Gaussian primitives by stabilizing early-stage optimization, enabling Fisher Information Matrix-based pruning. Pyramidal pruning (PP) removes additional redundant primitives learned under coarse supervision, while iterative pruning (IP) achieves the final compression with minimal additional degradation in visual quality.

6. CONCLUSION

In this work, we introduced a training framework that enables effective compression of 3D Gaussian Splatting [1]. Progressive pyramidal training places optimization into a low-residual regime at coarser pyramid levels, enabling Fisher Information Matrix (FIM)-based sensitivity estimates to remain reliable and enabling effective pruning of redundant Gaussian primitives throughout optimization. We further introduced a correlation-aware quantization strategy that exploits the distinct correlation structure of Gaussian parameters, achieving substantial storage reduction without compromising rendering quality.

A limitation of our approach is that it assumes coarser-level supervision produces sufficiently small residuals for a reliable FIM-based approximation. Exploring adaptive supervision schedules and alternative quantization strategies remains a promising direction for future work.

7. REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
- [2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [3] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, “Mip-nerf 360: Unbounded anti-aliased neural radiance fields,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5470–5479.
- [4] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park, “Compact 3d gaussian representation for radiance field,” *arXiv preprint arXiv:2311.13681*, 2023.
- [5] S. Girish, K. Gupta, and A. Shrivastava, “Eagles: Efficient accelerated 3d gaussians with lightweight encodings,” in *European Conference on Computer Vision*, 2024, pp. 54–71.
- [6] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang, “LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS,” in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., 2024, vol. 37, pp. 140138–140158.
- [7] A. Hanson, A. Tu, V. Singla, M. Jayawardhana, M. Zwicker, and T. Goldstein, “PUP 3D-GS: Principled Uncertainty Pruning for 3D Gaussian Splatting,” *arXiv preprint arXiv:2406.10219*, 2024.
- [8] S. Shin, J. Park, and S. Cho, “Locality-aware Gaussian Compression for Fast and High-quality Rendering,” in *International Conference on Learning Representations*, 2025.
- [9] X. Liu, X. and Wu, P. Zhang, S. Wang, Z. Li, and S. Kwong, “CompGs: Efficient 3d scene representation via compressed gaussian splatting,” in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, pp. 2936–2944.
- [10] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai, “HAC++: Towards 100X Compression of 3D Gaussian Splatting,” *arXiv preprint arXiv:2501.12255*, 2025.
- [11] U. Farooq, J. Guillemaut, G. Thomas, A. Hilton, and M. Volino, “Optimized 3D Gaussian Splatting using Coarse-to-Fine Image Frequency Modulation,” in *Proceedings of the 22nd ACM SIGGRAPH European Conference on Visual Media Production*. Dec. 2025, CVMP ’25, p. 1–10, ACM.
- [12] L. Goli, C. Reading, S. Sellán, A. Jacobson, and A. Tagliasacchi, “Bayes’ Rays: Uncertainty Quantification for Neural Radiance Fields,” 2023.
- [13] W. Jiang, B. BLei, and K. Daniilidis, “FisherRF: Active View Selection and Uncertainty Quantification for Radiance Fields using Fisher Information,” 2024.
- [14] A. Hanson, A. Tu, G. Lin, V. Singla, M. Zwicker, and T. Goldstein, “Speedy-Splat: Fast 3D Gaussian Splatting with Sparse Pixels and Sparse Primitives,” 2025.
- [15] K. P. Kumar and V. Reddi, “Significance of Spearman’s rank correlation coefficient,” *International Journal For Multidisciplinary Research*, vol. 5, no. 4, pp. 1–4, 2023.
- [16] R. T. Seeley, “Spherical Harmonics,” *The American Mathematical Monthly*, vol. 73, no. 4P2, pp. 115–121, 1966.
- [17] M. Moczulski, M. Denil, J. Appleyard, and N. de Freitas, “ACDC: A Structured Efficient Linear Layer,” 2016.
- [18] M. Huh, B. Cheung, P. Agrawal, and P. Isola, “Straightening Out the Straight-Through Estimator: Overcoming Optimization Challenges in Vector Quantized Networks,” in *Proceedings of the 40th International Conference on Machine Learning*, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds. 23–29 Jul 2023, vol. 202 of *Proceedings of Machine Learning Research*, pp. 14096–14113, PMLR.
- [19] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, “Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5470–5479.
- [20] A. Knapitsch, J. Park, Q. Zhou, and V. Koltun, “Tanks and temples: Benchmarking large-scale scene reconstruction,” *ACM Transactions on Graphics (ToG)*, vol. 36, no. 4, pp. 1–13, 2017.
- [21] P. Hedman, J. Philip, T. Price, J. Frahm, G. Drettakis, and G. Brostow, “Deep blending for free-viewpoint image-based rendering,” *ACM Transactions on Graphics (ToG)*, vol. 37, no. 6, pp. 1–15, 2018.
- [22] J. L. Schonberger and J. Frahm, “Structure-From-Motion Revisited,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.